

Correct Optimizations: Phase 1 Report

Based on Minotaur: A SIMD-Oriented Synthesizing Superoptimizer [1]

Kilian Schulz
Technical University Munich

Ilja Gamza
Technical University Munich

Tomás Gaspar
Technical University Munich

Claudia Dünzinger
Technical University Munich

Abstract

TODO

1 Introduction

This report aims to analyze and propose opportunities to continue the research from Liu et al. [1].

2 Paper Summary

2.1 Context

Minotaur is a superoptimizing compiler that identifies missed optimization opportunities in the LLVM compiler during a search of the program space. Its synthesizer creates improved machine code by using program synthesis, generating and comparing equivalent but cheaper instruction sequences. To guarantee correctness of the optimization, Alive2 was extended to formally verify the generated code. Additionally, to reduce computation overhead, Minotaur also caches discovered transformations for reuse in future compilations.

The use case of Minotaur lays in compilers that are already heavily optimized but still produce suboptimal code, particularly in SIMD operations. The authors demonstrated that with LLVM’s advanced autovectorizer output that produced redundant or inefficient SIMD instructions. In those missed opportunities, Minotaur corrects these suboptimal instruction sequences either by inserting improved machine code during the compilation or by suggesting the optimizations directly for the LLVM. Hence, the authors enabled higher performing and additionally formally verified machine code without hindering the compilers performance itself.

Before Minotaur, several superoptimizer already existed that search through the program code for more efficient sequences instead of performing set transformations like traditional compilers. Common examples include Souper – with focus on scalar integer optimizations and is now integrated in the LLVM – and STOKE, which uses stochastic search for

faster x86 code blocks. However, those tools only addressed part of the problem. Souper detects only improvements for integer operations, but not for floating-point or SIMD instructions. STOKE’s improvements include complex optimizations, but the tool relies on randomized testing and not formal verifications for its transformations. Therefore, the combined challenge of performing complex vector or floating point optimizations, formally verifying the transformation and integrating those capabilities into production compilers remained unsolved, until Minotaur.

Designing a efficient and high performing superoptimizer included several challenges. First, the search space of replacement options for a suboptimal code sequence is enormous, making an exhaustive search for the optimal choice computationally expensive. Second, the semantic formal and not random equivalency check adds further overhead – especially for hard to verify SIMD and floating point operations. Minotaur relies on SMT solvers for the code improvement checks, but these solvers often struggle with large complex expressions. At the same time a sufficient large number of lines of code is needed to provide enough context for determining improvement options in the first case. Furthermore, since Minotaur aims for low level, hardware aware optimizations, the authors required detailed information about pipelines and cache effects to accurately model the performance of the superoptimizer. These difficulties of realizing high performance without high computation overhead, required a multiple stages process.

2.2 Contributions

Because today’s SMT solvers cannot optimize entire LLVM functions, Minotaur instead optimizes each instruction individually. It therefore breaks the LLVM function in SSA form down into cuts, which are small a loop-free subsets of the the instruction’s dependencies, to capture where the values used in the instruction come from. The size of the cut hereby influences the optimization, as too small cuts contain not enough context to determine a better code sequence, while

too large cuts increase the solvers computation time. Through an empirical study the authors selected the cut size of four as it provides a good balance. For each cut, the SMT solver searches for a semantically equivalent but more efficient instruction sequence. These improvements are then verified with Alive2, which the authors extended to support SIMD and floating point operations. Lastly, for each optimization its speedup is estimated and its original and transformed code cached for reuse in future compilations.

To measure how Minotaur’s optimizations influence the overall compilations performance, the authors benchmarked Minotaur on the SPEC CPU 2017, GMP and libYUV on intel and amd hardware with LLVM 18 compiled at -O3. They determined a maximum speedup of 1 - 7 percent, where SIMD fragments made out 75 percent of the performance gains. When the compilation is performed with an empty cache, Minotaur increased the runtime significantly. With a filled cache, however, only an additional 3 percent of computation time was needed. Overall, the authors determined 15 simple and general floating-point and vector optimizations, which they then reported the original LLVM.

3 Artifact Evaluation Results

TODO

4 Research Ideas

TODO

Acknowledgments

TODO

Availability

TODO

References

- [1] Zhengyang Liu, Stefan Mada, and John Regehr. Minotaur: A SIMD-oriented synthesizing superoptimizer. *Proceedings of the ACM on Programming Languages (OOPSLA2)*, 8(OOPSLA2):1–25, October 2024. <https://doi.org/10.1145/3689766>.